LLMs as Temporal Point Processes: Token Likelihoods and Event Text

Md Rofiqul Islam
Baylor University, Department of Computer Science
Waco, Texas, USA

ABSTRACT

After a failed login, a security analyst may want to know how soon another alert will arrive and whether it will be an account lock. A temporal point process assigns probabilities to both outcomes as time passes. We show how a language model can do this using the probabilities it assigns to tokens. Each waiting time belongs to one interval. The probability of that interval, a rule for distributing probability inside it, and a token for the event type together define a continuous-time model. On simulated Hawkes sequences, the fitted model's negative log-likelihood was within 0.012 nats per event of the generating process. We then ask what changes when the model can read earlier Amazon reviews. Across two context budgets, the overall score showed no consistent gain. In one training run, reviews improved the conditional type score for rare categories and worsened it for frequent ones.

1 INTRODUCTION

Suppose a server records a successful login at 10:00:00 and failed logins at 10:01:15 and 10:01:30. A forecast of fifteen seconds leaves two practical questions open. How much longer could the operator reasonably wait? Which alert should the operator expect? Now suppose the last entry also says "unusual source IP." Could that detail change the forecast once the model knows the times and alert types?

A marked temporal point process (TPP) describes this kind of record. A *point* is an event at a particular time. Its *mark* is the associated label, such as a failed login. The *history* contains the information available before the forecast. An event record preserves both the order of the two failures and the fifteen seconds between them. A Hawkes process models clustering by letting an event temporarily raise the rate of later events [4]. Neural TPPs learn how history changes the rate or the distribution of waiting times [10, 11, 14].

A language model can also read the message attached to an alert. LAMP uses language to reason about candidate events proposed by another model [12]. TPP-LLM learns a separate network, called a head, that converts a language model's representation into an event rate [8]. Language-TPP writes time as four bytes and generates event attributes with the language model itself [6]. A byte representation specifies how to write a time. To score an irregular event stream we also need to say how much probability lies between recorded times and how likely a period of silence is.

Other approaches address different difficulties. FIM-PP learns from simulated processes and uses example histories to infer rates [1]. CDiff models several future events jointly [13]. Here we study one next event at a time. RQ1 asks whether token probabilities suffice to define its timing and type. RQ3 asks what descriptions add when the forecast targets stay the same.

2 RQ1. CAN THE LANGUAGE MODEL ITSELF DEFINE THE PROCESS?

2.1 From a token to a waiting time

Let H be the history, τ the wait until the next event, and k its mark. Divide the nonnegative time axis into ordered, nonoverlapping intervals called bins. The last bin covers every wait beyond a threshold A. The model names a finite bin by choosing a broad range, a narrower range inside it, and finally a small interval. At each choice we normalize the scores of the permitted tokens so their probabilities sum to one. The mark follows the time tokens.

Suppose the probabilities of the three choices leading to $[15, 16)$ seconds are 0.6, 0.5 and 0.2. Each choice uses the choices already made. The probability of that bin is their product, 0.06. If the model then assigns probability 0.7 to a lock, the chance of a lock in that second is 0.042. This extends the digit-bin idea in LLMTime [2] to a joint forecast of time and mark.

Write p_B for the probability of reaching bin B and $p_{k|B}$ for the mark probability there, both given H. Spread mass evenly through each finite bin of width w_B. In the tail use an exponential density of rate β. The continuous joint density is

$$f(\tau, k \mid H) = p_B p_{k|B} u_B(\tau), \quad u_B(\tau) = \begin{cases} 1/w_B & B \text{ finite}, \\ \beta e^{-\beta(\tau - A)} & \tau \geq A. \end{cases} \tag{1}$$

Density measures probability per unit of time. Spreading 0.042 over one second gives 0.042 per second. Spreading the same mass over two seconds gives 0.021 per second. The bin boundaries and tail rate are estimated from training waits and then held fixed. The uniform rule limits the detail the model can recover inside a bin.

2.2 What silence tells us

The survival probability $S(t \mid H)$ is the chance that the next wait exceeds t. Add the masses of all bins later than t and the unused fraction of the bin containing t. At 15.4 seconds, 60% of $[15, 16)$ remains. Its contribution is $0.06 \times 0.6 = 0.036$. Each later branch that separates from the observed path counts in full. For a specified wait, a single forward pass along its path supplies these branch probabilities. Computing the mean wait requires probabilities from other paths too.

Once the operator has waited t seconds, only outcomes that survived that long remain possible. Dividing by their total probability gives the instantaneous rate for mark k,

$$\lambda_k(t \mid H) = f(t, k \mid H)/S(t \mid H). \tag{2}$$

If half the forecasts survive to 15.4 seconds, the lock rate there is $0.042/0.5 = 0.084$ per second. Multiplying this rate by a sufficiently short interval approximates the conditional probability of a lock during that interval.

For a record ending a known time g after its last event, the log-likelihood sums the log densities of the observed events and adds $\log S(g \mid H_{\text{last}})$. A log turns the product of successive probabilities into a sum. If monitoring ends ten seconds after a login, we know the next wait exceeded ten seconds. This is a *right-censored* wait. A model that expects an immediate event pays a penalty for that observed silence. A break used to fit a sequence into computer memory supplies no such evidence by itself. In the experiments the first event provides context and scoring starts with the next event.

2.3 Assumptions and evidence

A finite tree with one path per bin and normalized probabilities at every branch assigns total mass one to its leaves. Each within-bin density integrates to one, so summing over marks and integrating over waits preserves that total. Repeated forecasts also need to rule out infinitely many events in finite time. Uniformly bounded token scores give the tail a positive minimum probability across histories. Before the tail, survival is therefore bounded away from zero and the smallest bin width bounds the density. In the tail the total rate equals β. These facts bound the rate and suffice to rule out explosion. Calibration is a separate empirical question about how well predicted probabilities match observed frequencies.

We adapted Qwen2.5-1.5B with LoRA, which trains small updates to the original weights, and learned vocabulary rows for the new tokens [5, 9]. Training fits the new tokens, scores observed times and marks, then adds terminal silence. Validation data set temperatures that adjust how concentrated token probabilities are. The main code uses three eight-way levels for 512 finite bins and a tail. Table 1 compares alternatives on a simulated three-type Hawkes process. Negative log-likelihood (NLL) measures the penalty assigned to observations. Lower is better. A nat uses natural logarithms.

Table 1: Synthetic Hawkes test, mean of three training seeds. Gap is excess NLL over the generating process in nats per scored event.

Model	NLL	Gap
Three-level time code	1.829	0.012
Two-level time code	1.831	0.013
Single-level code	1.886	0.068
Digit strings	1.826	0.008
GRU with mixture density	1.827	0.010
Transformer with mixture density	1.836	0.019

The digit code scored slightly better while using about five tokens per wait, compared with three for the main code. The generating process minimizes the expected log penalty. Its score on a finite sample is a reference value. On short Hawkes windows, adding silence during training reduced the terminal penalty from 0.243 to 0.189 nats per event and the overall penalty from 1.719 to 1.704 (two seeds). In short windows the waits that finish tend to be shorter than the ones cut off by the window end.

Real benchmarks. On StackOverflow the hierarchical model scored 2.186 nats per event versus 2.200 for the GRU. On Retweet the ordering reversed (6.345 versus 5.628). These scores depend on recording precision. On Taobao, tiny numerical differences between repeated waits allowed narrow bins and unusually high densities. A comparison on rounded records should score the probability each model assigns to the same recorded interval. Ties also require an explicit convention because the continuous model assigns zero probability to an exactly simultaneous pair.

Two other measurements answer narrower questions. The stored mark accuracy uses the observed time bin. Forecasting the mark before the event arrives requires averaging over possible times, $P(k \mid H) = \sum_B p_B p_{k|B}$. For time error on Taobao, the language model predicted means for 3,000 events and the baselines scored all 27,955. A direct comparison needs the same event IDs. We draw the empirical conclusion for RQ1 from the synthetic experiment, where the generating likelihood is known.

3 RQ3. WHEN DO EARLIER DESCRIPTIONS HELP?

A product category tells us that a customer reviewed a charger. The sentence “the connector stopped working after a week” adds a detail that the category alone cannot convey. Giving a model the name of the category tests a different idea. Every occurrence has the same name, so the words may help a pretrained model interpret an ID without revealing a new observation.

LAST SToP scrambles event names and finds a much larger change in type prediction than in time error [3]. Language-TPP removes descriptions in an Amazon experiment and reports a timing benefit from reading them [6]. LAMP tests reasoning over possible causes [12]. MM-TPP studies multimodal histories, where an image ablation measures visual information [7]. These experiments address different questions about which part of an input helps a forecast.

3.1 The Amazon comparison

We use the Cell Phones & Accessories subset of Amazon Reviews 2023. Each event is a posted review. Its time is the posting time, its mark is one of 24 product-category labels, and its text is the title and body truncated to 48 tokens. Eligible users have at least five reviews. Timestamp quantiles divide events into training, validation and test periods at the 80th and 90th percentiles. Earlier events remain available as history. A separate tenth of users is held out across their full timelines. The main FULL test run scored 12,849 events. A posting time measures when someone wrote about a product. Purchase time could differ substantially.

Each input predicts the same time and mark. NO-TEXT supplies earlier times and category IDs. NAMES appends a category name to each ID. SCRAMBLED appends an invented string of roughly the same character length, which can occupy a different number of tokens. FULL appends the event’s own review. SHUFFLED draws a substitute review of the same category. TEXT-ONLY masks each earlier mark from subsequent positions, leaving times and reviews visible. The event budget allows up to 32 events per window. The token budget allows roughly 1,024 tokens, so long reviews can displace older events.

We subtract the score with text from the score without it. Positive values favor text. Both conditions use the same time bins, so

their within-bin densities cancel in the difference. The gain splits into three parts: the change in log probability of the observed bin, the change for the observed mark given its bin, and the change for terminal silence. These terms describe where the fitted forecasts differ.

Table 2: Gain from FULL over NO-TEXT on Amazon, nats per event, one training seed. Positive entries favor reading reviews.

Score component	Event budget	Token budget
Time bin	+0.087	−0.036
Mark given bin	−0.016	+0.005
Terminal silence	−0.085	+0.038
Total	−0.014	+0.007

Under the event budget, reviews improved the score for observed waits by almost exactly as much as they worsened the score for quiet gaps (Table 2). The total changed sign under the token budget. Approximate 95% paired user-bootstrap half-widths for the totals were 0.013 and 0.012. Resampling users keeps reviews by the same person together. It measures variation across test users for the trained models. Variation across training runs remains unknown.

Which events changed? Group the 24 categories into thirds by training frequency. Under the event budget, the rarest third accounted for 4% of test events. Reading reviews improved the conditional mark score there by 0.137 nats per event (bootstrap half-width 0.072). The most frequent third accounted for 84% and lost 0.040 (half-width 0.008). Thus a gain for rare categories can coexist with a small aggregate loss. This exploratory comparison uses the category eventually observed. An analyst could not use it in advance to decide which incoming events deserve a text forecast.

The conditional mark gains were slightly negative in every history-length group, including users with only one earlier event. Similarity between neighboring reviews likewise requires the next review to have arrived. That analysis describes past errors. Removing earlier category IDs while retaining reviews worsened the total score by 0.088 and 0.098 nats under the two budgets. The explicit labels still helped in these runs.

Controls and uncertainty. Under the event budget FULL gained 0.004 nats over SHUFFLED. Adding scrambled strings to IDs changed the total by −0.036 nats under the event budget and +0.030 under the token budget. Extra tokens change sequence positions, history coverage and optimization. One run per condition cannot separate these effects from randomness in training. The shuffle routine can also draw outside the earlier-review pool when it finds no suitable donor. Category selection used frequencies from the full cohort before splitting. A prospective replication should fix the vocabulary from training data and enforce an earlier, different-user donor for every substitution.

Who contributes silence? The builder includes a user in a validation or test period only when the user posts during that period. Someone who stays silent throughout contributes nothing to its score. A second issue arises when the last review precedes the test boundary. Suppose the test begins 20 days after a review and the next one arrives on day 30. At the start of testing we already know the wait exceeded 20 days. The appropriate conditional density is $f(30, k \mid H)/S(20 \mid H)$. If monitoring ends on day 30 with no arrival, the contribution is $S(30 \mid H)/S(20 \mid H)$. The stored scores describe selected transitions and their terminal gaps. Estimating a full-period likelihood requires both the boundary correction and all eligible silent users.

Writing a review as an auxiliary task. Training the model also to write reviews with loss weight 0.1 lowered the event-budget score from 6.742 to 6.676 in one run. Generated descriptions were evaluated with the observed time bin and category supplied. Generation stops at a length cap, while training scores the closing token. We treat writing as an auxiliary objective. Low measured eight-word overlap with indexed training reviews (0.2–0.4% across the two writing variants) leaves open the question of pretraining recall.

4 CONCLUSION

The same branch probabilities that place a login alert in a time interval tell an operator how much probability remains after fifteen seconds of silence. This connects a token prediction to a continuous event rate and makes unfinished waits informative during training. The synthetic experiment tests that connection against a known process. On rounded operational logs, the next step is to score all models at the precision the recording system actually supports.

For reviews, the clearest difference was between rare and frequent observed categories. Historical text improved the conditional category score for the former in one run, while the aggregate benefit depended on the context budget and observation protocol. Before using descriptions to predict rare security alerts or clinical events, a study should repeat paired comparisons across seeds, test conditions identifiable from prior history, and include users who remain silent throughout follow-up. Evaluating reviews posted after the backbone's training cutoff would also help distinguish generalization from familiarity with public reviews.

Code availability. The implementation, experiment configurations and data-preparation scripts are available at https://github.com/Rofiqul-Islam/Headless_TPP.

Acknowledgments. Claude assisted with organizing notes, drafting text and writing experiment code. Codex assisted with revising the manuscript and checking the evaluation procedures against the project.

REFERENCES

[1] David Berghaus, Patrick Seifner, Kostadin Cvejoski, César Ojeda, and Ramsés J. Sánchez. 2026. In-Context Learning of Temporal Point Processes with Foundation Inference Models. In *International Conference on Learning Representations (ICLR)*.

[2] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. 2023. Large Language Models Are Zero-Shot Time Series Forecasters. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[3] Shubham Gupta, Thibaut Durand, Graham Taylor, and Lilian W. Białokozowicz. 2025. LAST SToP for Modeling Asynchronous Time Series. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 21297–21321.

[4] Alan G. Hawkes. 1971. Spectra of Some Self-Exciting and Mutually Exciting Point Processes. *Biometrika* 58, 1 (1971), 83–90.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.

[6] Quyu Kong, Yixuan Zhang, Yang Liu, Panrong Tong, Enqi Liu, and Feng Zhou. 2026. Byte-token Enhanced Language Models for Temporal Point Processes Analysis. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*. 7013–7023. https://doi.org/10.1145/3774904.3792197

[7] Jichu Li, Yilun Zhong, Zhiting Li, Feng Zhou, and Quyu Kong. 2026. Long-Range Modeling and Processing of Multimodal Event Sequences. In *International Conference on Learning Representations (ICLR)*.

[8] Zefang Liu and Yinzhu Quan. 2024. TPP-LLM: Modeling Temporal Point Processes by Efficiently Fine-Tuning Large Language Models. *arXiv preprint arXiv:2410.02062* (2024).

[9] Qwen Team. 2025. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2025).

[10] Oleksandr Shchur, Marin Biloš, and Stephan Günnemann. 2020. Intensity-Free Learning of Temporal Point Processes. In *International Conference on Learning Representations (ICLR)*.

[11] Oleksandr Shchur, Ali Caner Türkmen, Tim Januschowski, and Stephan Günnemann. 2021. Neural Temporal Point Processes: A Review. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI)*.

[12] Xiaoming Shi, Siqiao Xue, Kangrui Wang, Fan Zhou, James Y. Zhang, Jun Zhou, Chenhao Tan, and Hongyuan Mei. 2023. Language Models Can Improve Event Prediction by Few-Shot Abductive Reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[13] Mai Zeng, Florence Regol, and Mark Coates. 2024. Interacting Diffusion Processes for Event Sequence Forecasting. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (PMLR, Vol. 235)*. 58407–58430.

[14] Feng Zhou, Quyu Kong, Jie Qiao, Cheng Wan, Yixuan Zhang, and Ruichu Cai. 2026. Advances in Temporal Point Processes: Bayesian, Neural, and LLM Approaches. *Transactions on Machine Learning Research* (2026).